



Lab 10. Running Containers at Scale with Kubernetes

By the end of this lab exercise, you should be able to:

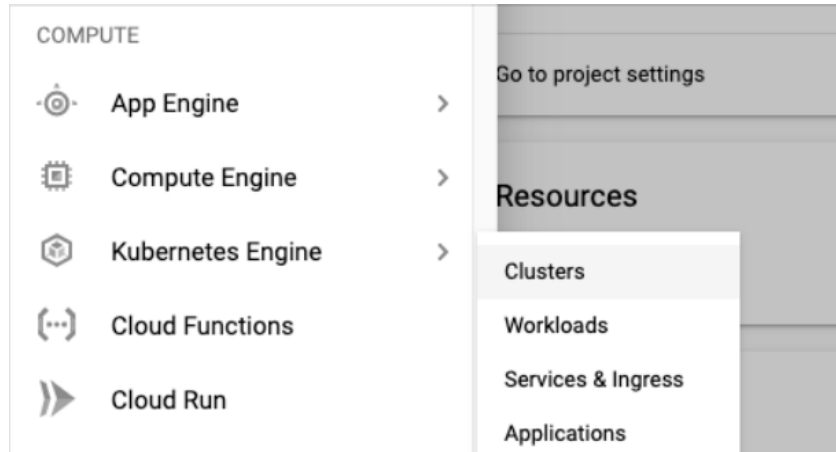
- Set up a managed Kubernetes environment with Google Cloud
- Set up a `kubectl` client, connect and work with the Kubernetes cluster
- Deploy applications to the Kubernetes cluster and expose them as services

Set Up Kubernetes Cluster with GCP

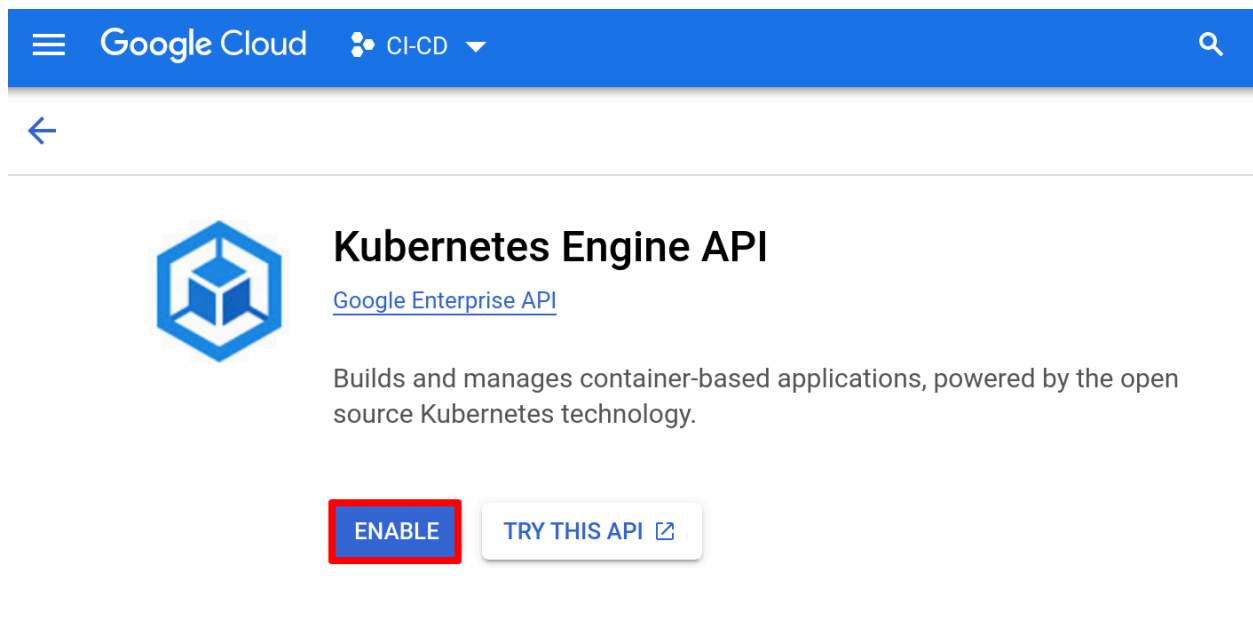
Set up a Google Cloud account by browsing to <https://cloud.google.com/free>.

Once setup, login to your account and browse to the cloud console by visiting <https://console.cloud.google.com/>.

From **Navigation** select **View All Products > Compute > Kubernetes Engine > Clusters**.



If it asks you to enable Kubernetes Engine API, do so by clicking on the **Enable** button.



You may also be asked to set up a Billing Account, which you should set up.

Billing required

Kubernetes Engine API requires a project with a billing account.

[CANCEL](#) [ENABLE BILLING](#)

Set the billing account for project “CI-CD”

Please select a **Google Cloud Platform** billing account to support this project. Some billing accounts may not be available. [Learn more](#)

Billing account *
My Billing Account ▼ ?

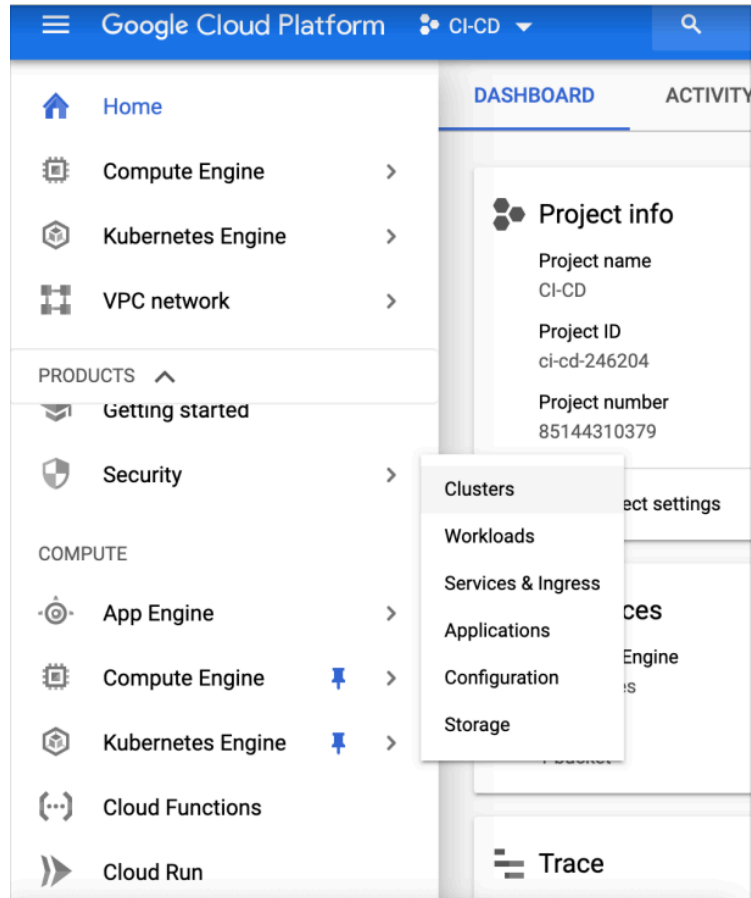
Any charges for this project will be billed to the account you select here.

CANCEL

SET ACCOUNT

Be aware that if you are signing up for a Free Trial with some free credits, enabling a billing account does not automatically charge you. When you run out of free credits, you will be given some time to allow Google Cloud to charge your credit card. Your resources may be deleted if you run out of free credits and do not allow Google Cloud to charge your credit card.

Once you have completed these steps, go to **Google Cloud Console**, select **Kubernetes Engine > Clusters** and you will arrive at the following page, ready to launch a cluster.



Kubernetes Engine

Kubernetes clusters

Containers package an application so it can easily be deployed to run in its own isolated environment. Containers are run on Kubernetes clusters. [Learn more](#)

CREATE

DEPLOY CONTAINER

TAKE THE QUICKSTART

Click on **Create Cluster**, choose **Standard**.

Google Cloud

LF Train-Cert CICD Course Dev

Search (/) for resou... Search

← Create an Autopilot cluster

SWITCH TO STANDARD CLUSTER

LEARN

- Cluster basics
 - Set up basics for your cluster
- Fleet registration
 - Manage multiple clusters together
- Networking
 - Define applications communication in the cluster
- Advanced settings
 - Review additional options
- Review and create
 - Review all settings and create your cluster

Cluster basics

Create an Autopilot cluster by specifying a name and region. After the cluster is created, you can deploy your workload through Kubernetes and we'll take care of the rest, including:

- ✓ **Nodes:** Automated node provisioning, scaling, and maintenance
- ✓ **Networking:** VPC-native traffic routing for public or private clusters
- ✓ **Security:** Shielded GKE Nodes and Workload Identity
- ✓ **Telemetry:** Cloud Operations logging and monitoring

Name

autopilot-cluster-1

Cluster names must start with a lowercase letter followed by up to 39 lowercase letters, numbers, or hyphens. They can't end with a hyphen. You cannot change the cluster's name once it's created.

CREATE CANCEL Equivalent REST or COMMAND LINE

This opens up a cluster configuration page similar to below. Provide the name for the cluster, e.g. **cd**, and select **Regular Channel** for control plane version. Do note, the actual version that you see may differ from what is shown in the screenshots.

Google Cloud

LF Train-Cert CICD Course Dev

Search (/) for resources, docs, products, and more Search

Kubernetes Engine / Create Kubernetes cluster

← Create a Kubernetes cluster

Cluster basics

- Fleet registration
- NODE POOLS
 - default pool
- CLUSTER
- Automation
- Networking
- Security
- Backup plan
- Metadata
- Features

Cluster basics

The new cluster will be created with the name, version, and in the location you specify here. After the cluster is created, name and location can't be changed.

Name

cd

Cluster names must start with a lowercase letter followed by up to 39 lowercase letters, numbers, or hyphens. They can't end with a hyphen. You cannot change the cluster's name once it's created.

Location type

Resource prices may vary between certain regions. [Learn more](#)

☒ Zonal

☐ Regional

Zone

us-west1-c

☐ Specify default node locations

Increase availability by selecting more than one zone

Current default: us-west1-c

Release channel

Select a release channel for GKE to pick versions for your cluster with your chosen balance between feature availability and stability. GKE automatically upgrades all clusters over time, even when no channel is selected. You can control the timing of upgrades through [maintenance policies](#) in cluster automation settings. [Learn about release channels](#)

Regular (recommended)

Kubernetes version

1.32.4-gke.1353003 (default)

Versions in the Regular channel have been qualified over a longer period. They offer a balance of feature availability and release stability. We recommend the Regular channel for most users. For known issues and workarounds, review [release notes](#).

Estimated monthly cost [Preview](#)

\$176.38

That's about \$0.24 per hour

Pricing is based on the resources you use, management fees, discounts and credits. [Learn more](#)

Switch to Autopilot cluster

LEARN

Create Cancel Code equivalent

Click on **Create** and wait for the cluster to be ready.

Connecting to Your Cluster

There are two ways to connect to your cluster. Either way will end up with you interacting with your cluster via a terminal in which you will run the commands given.

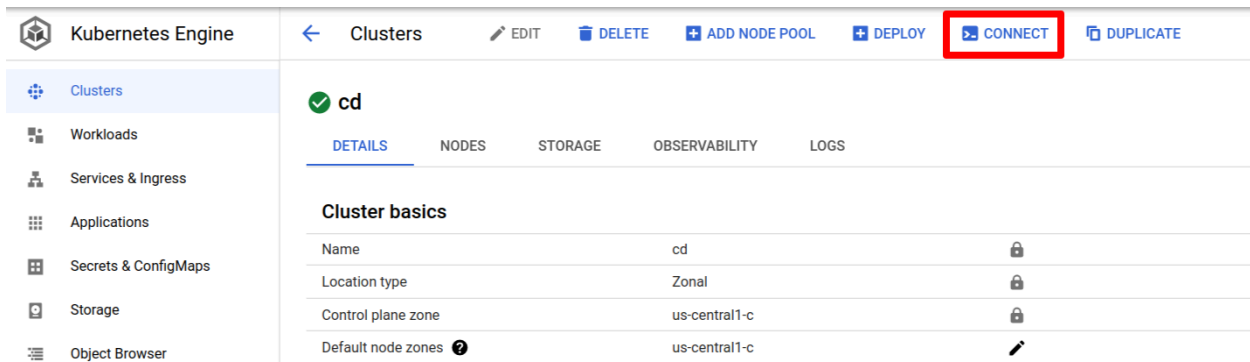
The first and easier way to connect to your cluster is via the **RUN IN CLOUD SHELL** option. The second is to manually connect to your cluster via your own machine.

We will go over both but it is suggested you use the easier method.

Connect to Your Cluster Via **RUN IN CLOUD SHELL**

Navigate to **Kubernetes Engine > Clusters > click on your cluster name**, as follows:

In the horizontal menu across the top of the screen you will see the **CONNECT** option. Click it:



The screenshot shows the Google Cloud Kubernetes Engine console. On the left is a sidebar with navigation options: Clusters (selected), Workloads, Services & Ingress, Applications, Secrets & ConfigMaps, Storage, and Object Browser. The main header area includes 'Kubernetes Engine' and a breadcrumb 'Clusters'. Action buttons include EDIT, DELETE, ADD NODE POOL, DEPLOY, CONNECT (highlighted with a red box), and DUPLICATE. Below the header, a cluster named 'cd' is selected, with tabs for DETAILS (active), NODES, STORAGE, OBSERVABILITY, and LOGS. The 'Cluster basics' section displays a table with the following information:

Cluster basics		
Name	cd	🔒
Location type	Zonal	🔒
Control plane zone	us-central1-c	🔒
Default node zones	us-central1-c	✎

A pop up window will appear. Click **RUN IN CLOUD SHELL**:

Connect to the cluster

You can connect to your cluster via command-line or using a dashboard.

Command-line access

Configure [kubectl](#) command line access by running the following command:

```
$ gcloud container clusters get-credentials cd --zone us-central1-c --project ci-cd-349400
```

RUN IN CLOUD SHELL

Cloud Console dashboard

You can view the workloads running in your cluster in the Cloud Console [Workloads dashboard](#).

[OPEN WORKLOADS DASHBOARD](#)

OK

A terminal will appear with the **gcloud** command to connect to your cluster pre-populated.

The screenshot shows the Google Cloud Console interface. On the left is a navigation menu with options like Clusters, Workloads, Services & Ingress, Applications, Secrets & ConfigMaps, Storage, Marketplace, and Release Notes. The main panel displays the details for a Kubernetes Engine cluster named 'cd'. The 'DETAILS' tab is active, showing a table of cluster basics:

Cluster basics		
Name	cd	🔒
Location type	Zonal	🔒
Control plane zone	us-central1-c	🔒
Default node zones	us-central1-c	✎
Release channel	Rapid channel	✎ UPGRADE AVAILABLE
Version	1.24.4-gke.800	
Total size	3	①
Endpoint	34.28.113.186	🔒

Below the cluster details, there is a 'CLOUD SHELL' terminal window. The terminal shows the command `gcloud container clusters get-credentials cd --zone us-central1-c --project ci-cd-349400` pre-populated and highlighted with a red box. The terminal output includes a welcome message and instructions on how to use the shell.

Press **Enter** and you will be connected to your cluster via a terminal.

Manual Connection to Your Cluster

To connect to your cluster via your own machine, use the following:

- Read the [Google SDK installation guide](#).
- Install Google SDK with [interactive option](#).
- Install [kubecti](#) by referring to the documentation relevant to your OS.

Navigate to **Kubernetes Engine > Clusters** and click on the name of your cluster.

OVERVIEW

OBSERVABILITY

COST OPTIMIZATION

Filter Enter property name or value

☐	Status	Name	Location	Number of nodes	Total vCPUs
☐		cd	us-central1-c	3	6

Click on **CONNECT**. If you don't see it, you are likely zoomed in too far in your browser settings and it will be under the vertical three-dot menu on the right.

← Clusters
 EDIT
DELETE
ADD NODE POOL
DEPLOY
CONNECT
DUPLICATE

✓ cd

DETAILS
 NODES
 STORAGE
 OBSERVABILITY
 LOGS

Cluster basics

Name	cd	🔒
Location type	Zonal	🔒
Control plane zone	us-central1-c	🔒
Default node zones ?	us-central1-c	✎
Release channel	Rapid channel	✎ UPGRADE AVAILABLE
Version	1.24.4-gke.800	
Total size	3	ⓘ
Endpoint	34.28.113.186	🔒
	Show cluster certificate	

A popup window should appear. Copy the `connect to the cluster` command and run it on the host where you have installed `kubectl`.

Connect to the cluster

You can connect to your cluster via command-line or using a dashboard.

Command-line access

Configure [kubectl](#) command line access by running the following command:

```
$ gcloud container clusters get-credentials cd --zone us-central1-c --project ci-cd-3494
```

[RUN IN CLOUD SHELL](#)

Verify `kubectl` configuration using the following commands:

```
kubectl version -o yaml
```

```
clientVersion:
  buildDate: "2025-05-15T20:09:34Z"
  compiler: gc
  gitCommit: 4cb5f0764b8d5f425645c6f433394fede34a8a26
  gitTreeState: clean
```

```
gitVersion: v1.32.4-dispatcher
goVersion: go1.23.8
major: "1"
minor: 32+
platform: linux/amd64
kustomizeVersion: v5.5.0
serverVersion:
  buildDate: "2025-05-19T04:19:46Z"
  compiler: gc
  gitCommit: d05b573072d7732178c3b9a376809c1d945f669f
  gitTreeState: clean
  gitVersion: v1.32.4-gke.1353003
  goVersion: go1.23.8 X:boringcrypto
  major: "1"
  minor: "32"
  platform: linux/amd64
```

kubectl get nodes

When you run `kubectl get nodes` you should see three nodes listed with **Ready** status.

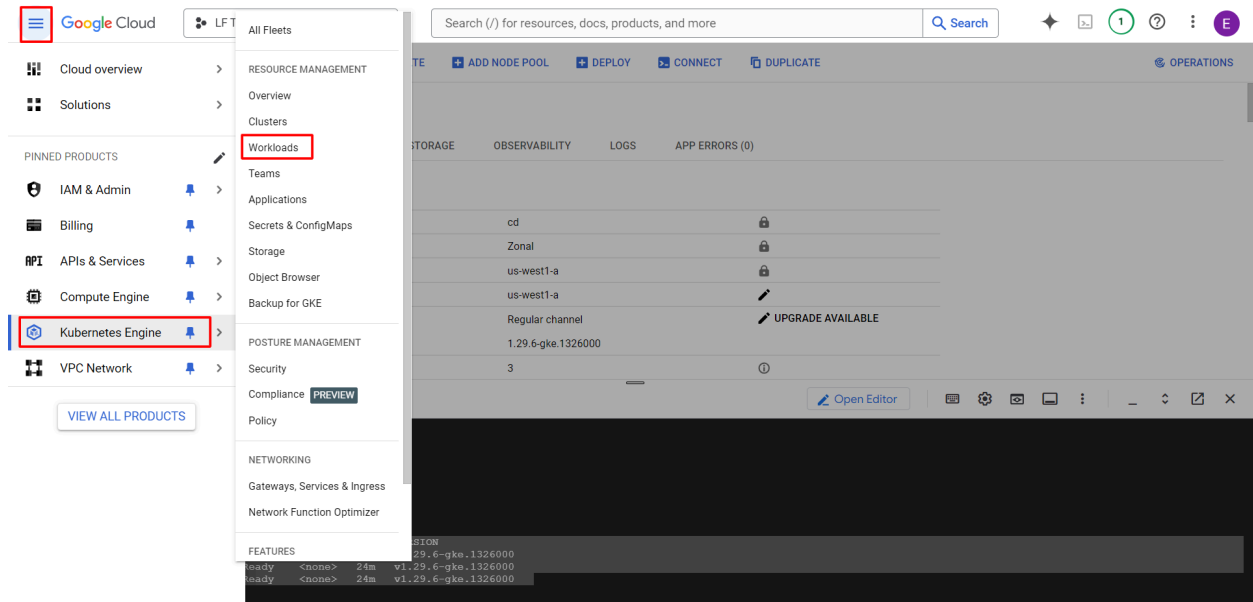
Sample output:

NAME	STATUS	ROLES	AGE	VERSION
gke-cd-default-pool-4c6b87d8-39cg	Ready	<none>	26m	v1.32.4-gke.1353003
gke-cd-default-pool-4c6b87d8-9rbn	Ready	<none>	26m	v1.32.4-gke.1353003
gke-cd-default-pool-4c6b87d8-pwg0	Ready	<none>	26m	v1.32.4-gke.1353003

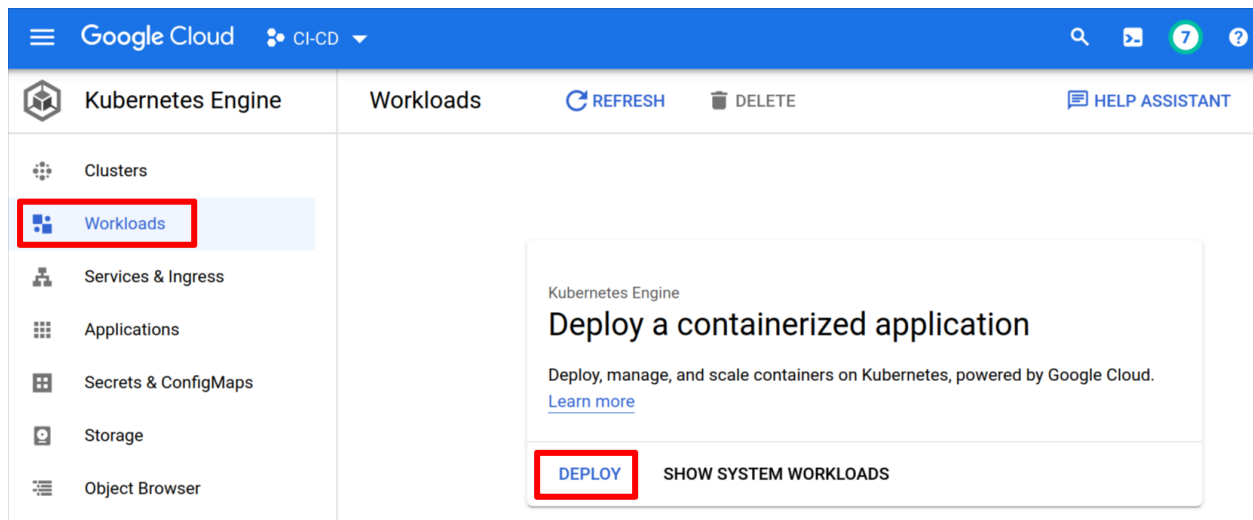
This validates that the cluster is ready and operational.

Deploy the Frontend Vote App with Kubernetes

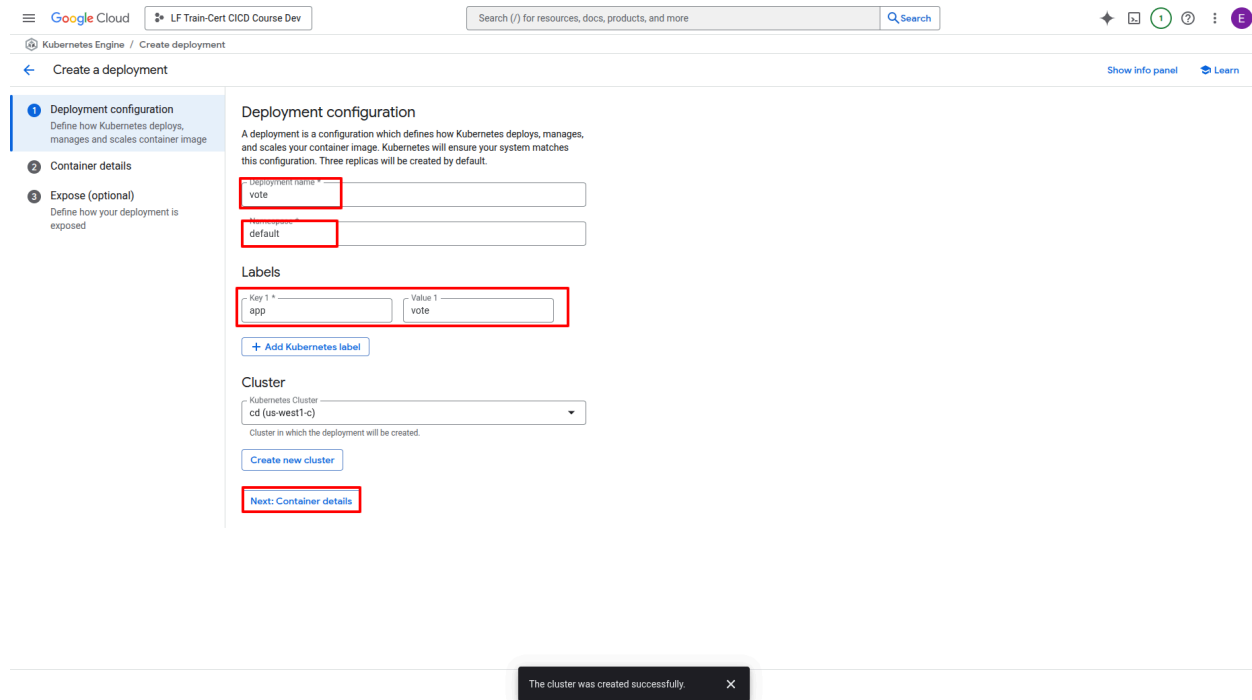
Navigate to Google Kubernetes Engine's **Workloads**:



Click **Create Deployment**.



Provide the configurations to deploy the frontend `vote` app as follows and click **Next** to access the container details page.



Google Cloud | LF Train-Cert CICD Course Dev | Search (/) for resources, docs, products, and more | Search

Kubernetes Engine / Create deployment

Create a deployment

Show info panel | Learn

1 Deployment configuration

Define how Kubernetes deploys, manages and scales container image

2 Container details

3 Expose (optional)

Define how your deployment is exposed

Deployment configuration

A deployment is a configuration which defines how Kubernetes deploys, manages, and scales your container image. Kubernetes will ensure your system matches this configuration. Three replicas will be created by default.

Deployment name *
vote

Namespace
default

Labels

Key 1 *
app

Value 1
vote

+ Add Kubernetes label

Cluster

Kubernetes Cluster
cd (us-west1-c)

Cluster in which the deployment will be created.

Create new cluster

Next: Container details

The cluster was created successfully

We will use our existing container image: `DockerHubUserID/vote`. NOTE: Replace the image with your actual image tag.

Google Cloud | LF Train-Cert CICD Course Dev | Search (/) for resources, docs, products, and more | Search

Kubernetes Engine / Create deployment

Create a deployment

Deployment configuration
Define how Kubernetes deploys, manages and scales container image

Container details

Expose (optional)
Define how your deployment is exposed

Container details

New container

Existing container image

New container image

nginx:alpine

eeaganiff/vote:latest

Select

Enter the name of any public image from Docker Hub, such as nginx:latest.
[Explore Docker Hub](#)

Environment variables

+ Add environment variable

Initial command

Overrides the default endpoint of the container image.

Done

Add a container

Back

Next: Expose (optional)

Deploy

Cancel

View YAML

Click **Deploy**.

vote

Using an existing cluster: us-central1-a/ci-cd

Using an existing cluster: us-central1-a/ci-cd

Creating a Deployment

Waiting for Pods

[HIDE ALL STEPS](#)

✓ vote

📘 To let others access your deployment, expose it to create a service

OVERVIEW

DETAILS

REVISION HISTORY

EVENTS

LOGS

YAML

CPU



Memory



Cluster	cd
Namespace	default
Labels	app: vote
Logs	Container logs , Audit logs
Replicas	3 updated, 3 ready, 3 available, 0 unavailable
Pod specification	Revision 1, containers: vote-1
Horizontal Pod Autoscaler	⚠️ Unable to read all metrics

Active revisions

Revision	Name	Status	Summary	Created on	Pods running/Pods total
1	vote-6f9c8b4586	✓ OK	vote-1: okapetanos/vote:latest	Nov 6, 2022, 6:55:33 AM	3/3

Managed pods

Revision	Name	Status	Restarts	Created on
1	vote-6f9c8b4586-7hqw7	✓ Running	0	Nov 6, 2022, 6:55:34 AM
1	vote-6f9c8b4586-6rvvr	✓ Running	0	Nov 6, 2022, 6:55:34 AM
1	vote-6f9c8b4586-kj2r2	✓ Running	0	Nov 6, 2022, 6:55:34 AM

Your app is now deployed on the cluster.

Pod Operations

From the host where `kubect1` is configured, you can explore pod operations by running the following commands:

```
kubect1 get pods
```

```
kubect1 get pods -o wide
```

NOTE: Replace `vote-xxxx` with the actual pod names displayed under `kubect1 get pods`.

```
kubect1 describe pods vote-xxxx
```

```
kubect1 logs vote-xxxx
```

```
kubectl exec vote-xxxx -- ps
```

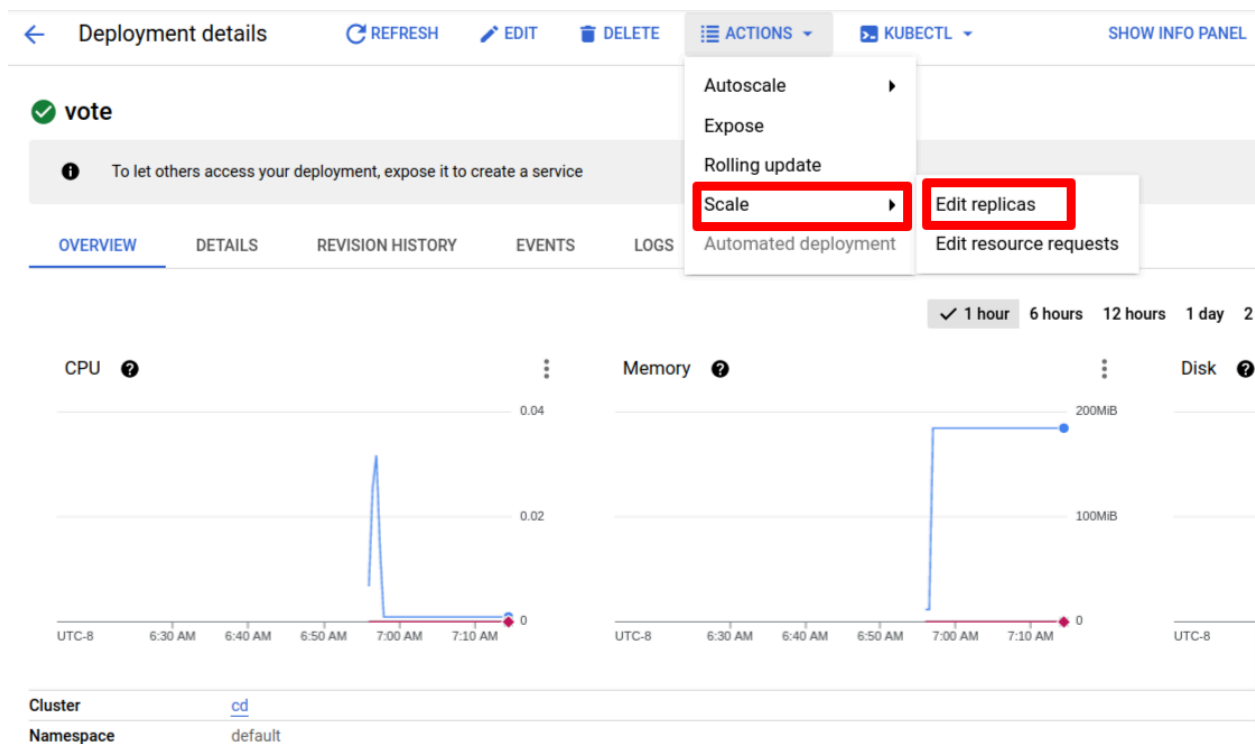
```
kubectl exec -it vote-xxxx -- sh
```

Use **CTRL + d** to exit from the container's shell.

Scalability

Scaling Manually

Click the **ACTIONS** menu and select **Scale > Edit replicas**.



Scale

Scale a workload to a new size.

Replicas *

* Indicates required field

CANCEL
SCALE

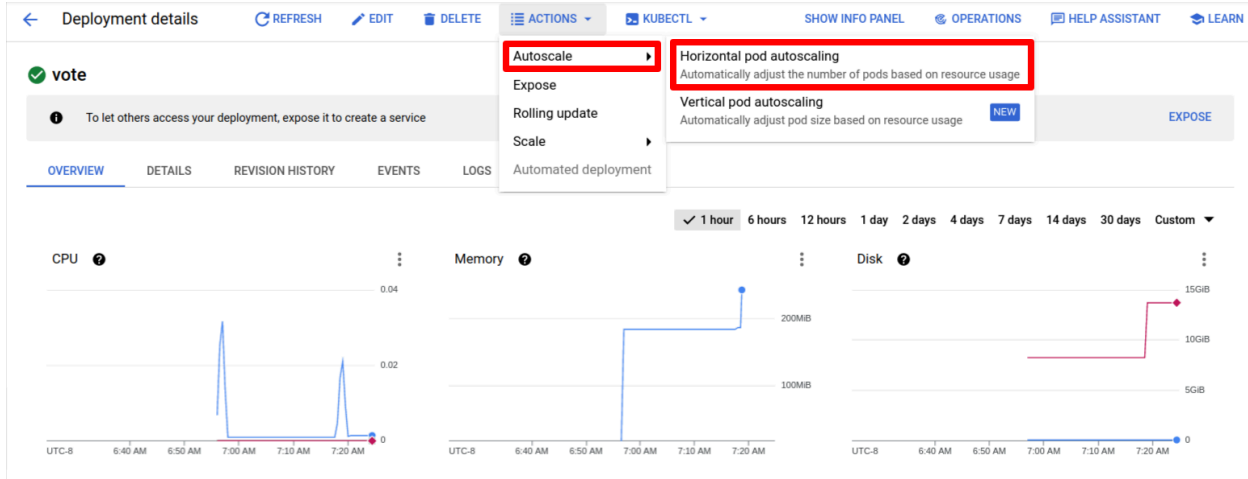
Under the **Managed pods** section, you will now see that 5 pods have been deployed.

Managed pods				
Revision	Name	Status	Restarts	Created on ↑
1	vote-6f9c8b4586-7hqw7	✓ Running	0	Nov 6, 2022, 6:55:34 AM
1	vote-6f9c8b4586-6rvvr	✓ Running	0	Nov 6, 2022, 6:55:34 AM
1	vote-6f9c8b4586-kj2r2	✓ Running	0	Nov 6, 2022, 6:55:34 AM
1	vote-6f9c8b4586-bklf8	✓ Running	0	Nov 6, 2022, 7:17:48 AM
1	vote-6f9c8b4586-8trk5	✓ Running	0	Nov 6, 2022, 7:17:48 AM

You have manually scaled your application using Kubernetes with Google's Kubernetes Engine.

Autoscaling

You will now configure Kubernetes to handle the scaling for your cluster automatically. Once again, navigate to **ACTIONS**. This time select **Autoscale > Horizontal pod autoscaling** from the dropdown, as follows:



Give your deployment a minimum of 2 replicas and a maximum of 8, as follows:

Configure Horizontal Pod Autoscaler

Horizontal Pod autoscaling increases and decreases the number of replicated pods to maintain performance and minimize cost. [Horizontal Pod Autoscaling](#)

Minimum number of replicas

2

Maximum number of replicas *

8

Autoscaling metrics

Use metrics to determine when to autoscale the deployment

CPU (80%)

[ADD METRIC](#)

* Indicates required field

CANCEL

DELETE

SAVE

You will now test that your autoscaling works by attempting to delete pods.

Availability

Try deleting a few pods after listing them and observe what happens. You should see new pods created automatically.

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
vote-799cf486bf-ctvww	1/1	Running	0	4s
vote-799cf486bf-jmr5b	1/1	Running	0	8s
vote-799cf486bf-jnwbs	1/1	Running	0	8s
vote-799cf486bf-p5wzc	1/1	Running	0	8s

NOTE: Replace the following **vote-xxxx-xxxx** pod names with your own.

```
$ kubectl delete pods vote-799cf486bf-jnwbs vote-799cf486bf-p5wzc
pod "vote-799cf486bf-jnwbs" deleted
pod "vote-799cf486bf-p5wzc" deleted
```

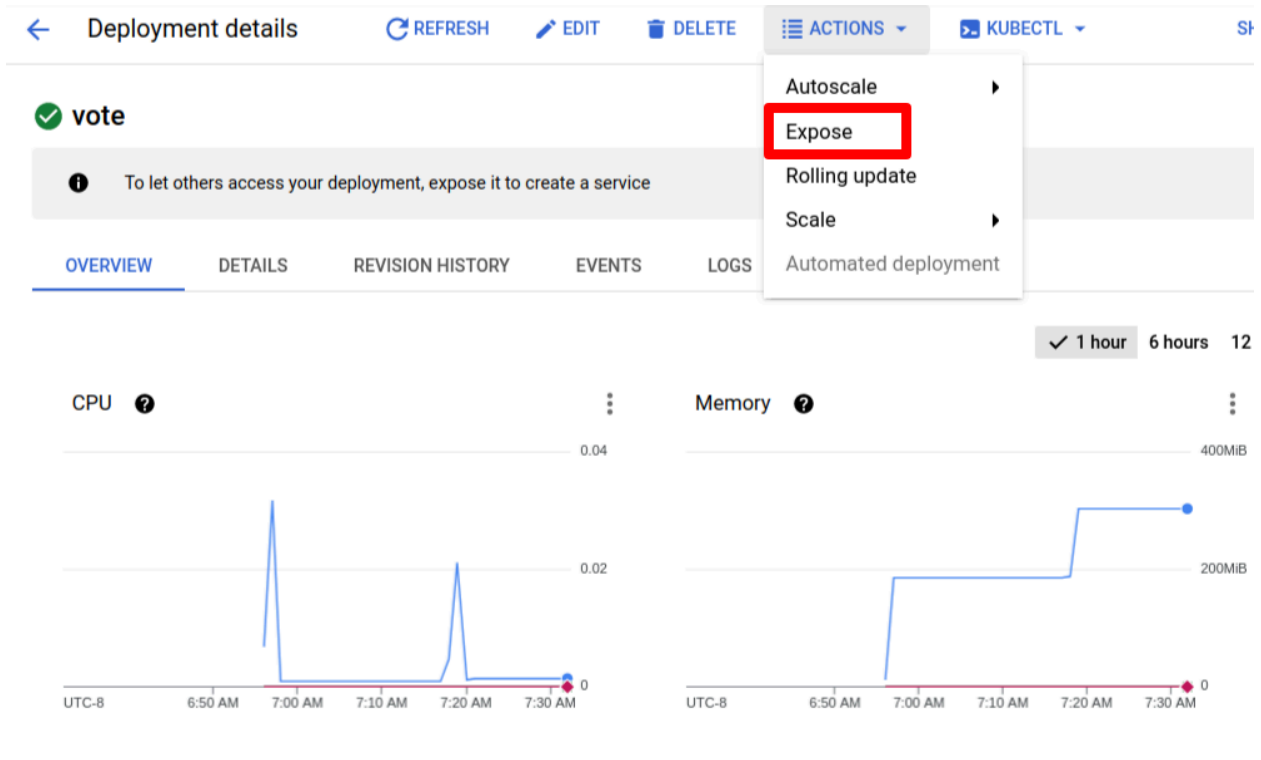
```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
vote-799cf486bf-bw4bf	1/1	Running	0	14s
vote-799cf486bf-ctvww	1/1	Running	0	34s
vote-799cf486bf-d7dzq	1/1	Running	0	14s
vote-799cf486bf-jmr5b	1/1	Running	0	38s

Kubernetes has created new pods to replace the ones you deleted. Kubernetes always maintains the desired scale that you provide and constantly monitors and takes action if the desired count changes. This is an example of availability, fault tolerance, and resilience.

Publishing and Load Balancing with Services

Browse to **Workloads > vote > Actions** and then select **Expose**.



Provide values for “Port” and “Target Port” as 80 and then choose **NodePort** from the **Service type** dropdown. This is an important step in order to access this application from outside the cluster.

Expose

Expose a resource's Pods using a Kubernetes Service.

Port mapping

Port 1
80

Target port 1
80

Protocol 1
TCP

+ ADD PORT MAPPING

Service type
Node port

* Indicates required field

CANCEL

EXPOSE

Once created, browse to **Kubernetes Engine > Gateways, Services & Ingress > Services > vote-xxxx**. Scroll to the **Ports** section and find the NodePort associated with it.

Serving pods

Name	Status	Endpoints	Restarts	Created on ↑
vote-6f9c8b4586-7hqw7	✓ Running	10.32.0.7	0	Nov 6, 2022, 6:55:34 AM
vote-6f9c8b4586-6rvvr	✓ Running	10.32.1.6	0	Nov 6, 2022, 6:55:34 AM
vote-6f9c8b4586-kj2r2	✓ Running	10.32.2.4	0	Nov 6, 2022, 6:55:34 AM
vote-6f9c8b4586-bklf8	✓ Running	10.32.2.5	0	Nov 6, 2022, 7:17:48 AM
vote-6f9c8b4586-8trk5	✓ Running	10.32.0.8	0	Nov 6, 2022, 7:17:48 AM

Ports

Port	Node Port	Target Port	Protocol
80	32245	80	TCP

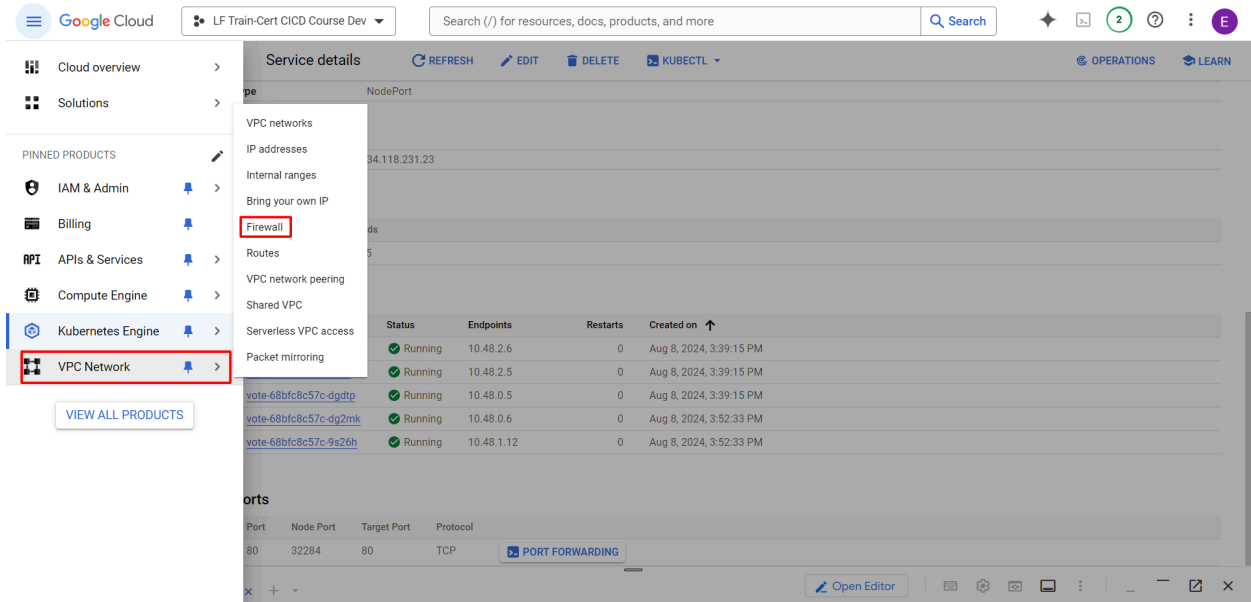
PORT FORWARDING

You will need to know this later.

Opening Firewall Rules

You may have already performed the following steps at the time of creating the cluster. If not, open the firewall rules, as follows, in order for you to be able to access the services from outside the cluster (e.g. using a browser on your workstation, which is outside the GKE cluster).

Navigate to the main cloud console menu and **VPC network > Firewall**:



Scroll to the rule that ends in **-a11** and click on it:

Filter Enter property name or value											
☐	Name	Type	Targets	Filters	Protocols / ports	Action	Priority	Network	Logs	Hit count	
☐	gke-cd-3050e07c-inkubelet	Ingress	gke-cd-3050e07c-inkubelet	IP ranges: 10.0.0.0/24	tcp:10255	Allow	999	default	Off	—	▼
☐	gke-cd-3050e07c-exkubelet	Ingress	gke-cd-3050e07c-exkubelet	IP ranges: 10.0.0.0/24	tcp:10255	Deny	1000	default	Off	—	▼
☐	default-allow-http	Ingress	http-server	IP ranges: 0.0.0.0/0	all	Allow	1000	default	Off	—	▼
☐	default-allow-https	Ingress	https-server	IP ranges: 0.0.0.0/0	tcp:443	Allow	1000	default	Off	—	▼
☐	gke-cd-3050e07c-a11	Ingress	gke-cd-3050e07c-a11	IP ranges: 10.0.0.0/24	tcp,udp,icmp,esp,ah	Allow	1000	default	Off	—	▼
☐	gke-cd-3050e07c-vms	Ingress	gke-cd-3050e07c-vms	IP ranges: 10.0.0.0/24	tcp:1-65535,udp:1-65535,icmp	Allow	1000	default	Off	—	▼

Click **EDIT**:

 Firewall rule details  **EDIT**  **DELETE**

gke-cd-3050e07c-all

Logs 
Off
[view in Logs Explorer](#)

Network
default

Priority
1000

Direction
Ingress

Action on match
Allow

Scroll to **Source IPv4 ranges** and enter 0.0.0.0/0:

Direction
Ingress

Action on match
Allow

Targets
Specified target tags

Target tags *
gke-cd-3050e07c-node

Source filter
IPv4 ranges

Source IPv4 ranges *
10.32.0.0/14 0.0.0.0/0 for example, 0.0.0.0/0, 192.168.2.0/24

Second source filter
None

Click **Save** at the bottom of the screen.

Find the external IP address of the nodes participating in the Kubernetes cluster.

☐	☒	gke-cd-default-pool-8fd7c0bf-g5kq	us-central1-c	gke-cd-default-pool-8fd7c...	10.128.0.22 (nic0)	35.238.107.163 (nic0)	SSH	⋮
☐	☒	gke-cd-default-pool-8fd7c0bf-g43h	us-central1-c	gke-cd-default-pool-8fd7c...	10.128.0.21 (nic0)	34.71.167.239 (nic0)	SSH	⋮
☐	☒	gke-cd-default-pool-8fd7c0bf-r8hn	us-central1-c	gke-cd-default-pool-8fd7c...	10.128.0.20 (nic0)	104.154.246.28 (nic0)	SSH	⋮

Alternately, you can find the external IPs of the nodes by using the following command:

```
kubectl get nodes -o wide
```

Sample output:

```

NAME                                STATUS    ROLES    AGE   VERSION
INTERNAL-IP  EXTERNAL-IP  OS-IMAGE
KERNEL-VERSION  CONTAINER-RUNTIME

gke-cd-default-pool-14c6b87d8-39cg  Ready    <none>   19h
v1.32.4-gke.1353003  10.138.0.38  34.53.78.232  Container-Optimized OS
from Google  6.6.72+      containerd://1.7.24

gke-cd-default-pool-14c6b87d8-9rbn  Ready    <none>   19h
v1.32.4-gke.1353003  10.138.0.37  34.168.221.251  Container-Optimized OS

```

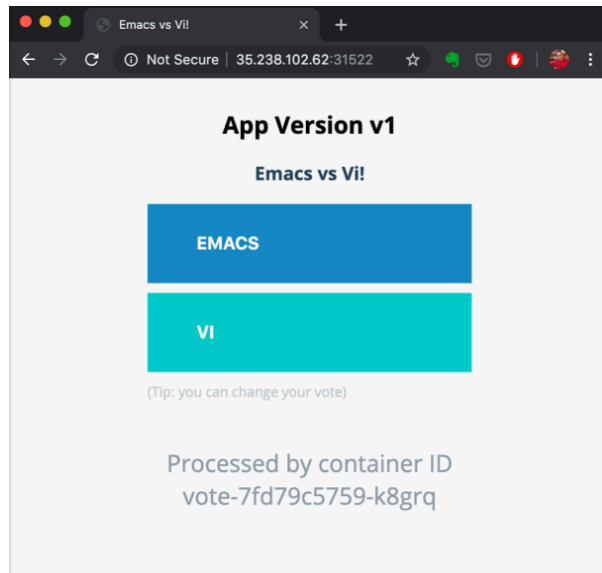
```

from Google    6.6.72+          containerd://1.7.24

gke-cd-default-pool-4c6b87d8-pwg0    Ready    <none>    19h
v1.32.4-gke.1353003    10.138.0.39    35.185.198.94    Container-Optimized OS
from Google    6.6.72+          containerd://1.7.24

```

Use any of the external IPs and load the application by using NodePort as `http://IPADDRESS:NODEPORT`. For example:



Try reloading the URL a few times and observe the container ID. Do you see the container IDs changing? If yes, the load balancing is in place.

Try clicking an option. You will notice that the app crashes. This is because the `redis` backend is not set up. This is what you will do in the next section.

Service Discovery

To launch the `redis` backend microservice go to **Kubernetes Engine > Workloads** and select the **Deploy** option.

Workloads REFRESH **DEPLOY** DELETE

Cluster ▼

Namespace ▼

RESET

SAVE

Workloads are deployable units of computing that can be created and managed in a cluster.

OVERVIEW COST OPTIMIZATION

Filter Is system object : False ✕ Filter workloads

☐	Name ↑	Status	Type	Pods	Namespace	Cluster
☐	vote	✓ OK	Deployment	5/5	default	cd

Enter the **Application name** and **Value 1** as `redis`.

2 Configuration

A deployment is a configuration which defines how Kubernetes deploys, manages, and scales your container image. Kubernetes will ensure your system matches this configuration.

Application name *
redis

Namespace *
default

Labels

Use Kubernetes labels to control how workloads are scheduled to your nodes. Labels are applied to all nodes in this node pool and cannot be changed once the cluster is created.

Key 1 *
app

Value 1
redis

Provide the image `redis:alpine` and click on **Deploy**.

The screenshot shows the Google Cloud console interface for creating a Kubernetes deployment. The breadcrumb navigation indicates 'Kubernetes Engine / Create deployment'. The left sidebar shows three steps: 'Deployment configuration' (checked), 'Container details' (active), and 'Expose (optional)'. The main panel is titled 'Container details' and contains a 'New container' section. In this section, the 'Existing container image' radio button is selected. The 'Image path' text input field contains 'redis:alpine'. Below this, there is a section for 'Environment variables' with an '+ Add environment variable' button, and an 'Initial command' text input field. At the bottom of the 'New container' section is a 'Done' button. Below the 'New container' section is an 'Add a container' button. At the very bottom of the page, there are three buttons: 'Deploy' (highlighted in blue), 'Cancel', and 'View YAML'.

Wait for the `redis` deployment to be ready.

Enter the following command into your terminal to expose the `redis` application as a service:

```
$ kubectl expose deployment redis --port=6379 --name=redis
```

```
service/redis exposed
```

Navigate to **Kubernetes Engine > Services and Ingress**. You will notice the `redis` service is listed:

The screenshot shows the 'Services & Ingress' page in the Google Cloud console. The left sidebar has 'Services & Ingress' highlighted with a red box. The main panel shows a table of services. The 'redis' service is highlighted with a red box. The table has columns: Name, Status, Type, Endpoints, Pods, Namespace, and Clusters. The 'redis' service is of type 'Cluster IP' with status 'OK'.

Name	Status	Type	Endpoints	Pods	Namespace	Clusters
redis	OK	Cluster IP	10.36.9.254	3/3	default	cd
vote-2t88g	OK	Node Port	10.36.10.80:80 TCP	5/5	default	cd

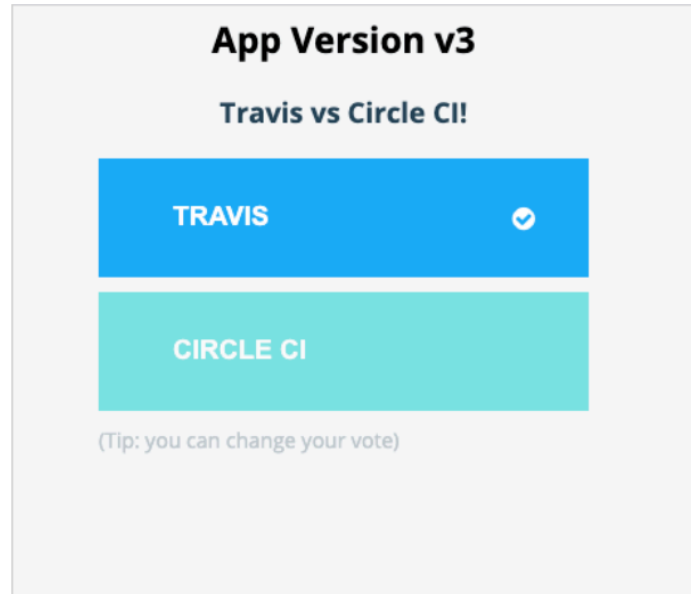
This means you can submit a vote on the vote application and it won't crash.

Run:

```
$ kubectl get node -o wide
```

The output will show your nodes. Under the **EXTERNAL-IP** column you will find the public IP's associated with each of your nodes. You can navigate to any of these IP's with the frontend port, which is found at **Kubernetes Engine > Services & Ingress > vote-xxxx > Ports > Node Port**. Enter the following into your web browser to see the frontend and click to vote:

NODEIPADDRESS:NODEPORT



As soon as you submit the vote, you should see a tick, which validates the communication between the frontend vote and the backend redis applications.

Summary

In this lab, we set up an account on Google Cloud. We installed the `kubect1` command line utility and set it up to interact with our cluster on Google Cloud. We then deployed the voting application to a Kubernetes Cluster on Google Cloud so that we can visit the app from any browser and submit a vote.